

Análise Computacional de Problemas de Scheduling por Programação Inteira Mista em JuMP

Gabriel de Moraes Rezende^a

^a*Programa de Pós-Graduação em Engenharia Elétrica, Universidade Federal do Amazonas, Manaus, Amazonas, Brasil*

Abstract

Este artigo apresenta uma análise computacional de três problemas clássicos de escalonamento: *Machine Scheduling*, *Job Shop Scheduling* e *Flow Shop Scheduling*. Os modelos foram formulados como problemas de Programação Linear Inteira Mista (MILP), implementados em Julia com JuMP e resolvidos com o solver HiGHS. A metodologia considera variáveis contínuas para os tempos de início e conclusão das tarefas, além de variáveis binárias para representar decisões de precedência e impedir sobreposição em máquinas compartilhadas. Para a instância de *Machine Scheduling*, o objetivo foi minimizar o atraso total, resultando em atraso total ótimo igual a 16 unidades de tempo. Para as instâncias de *Job Shop* e *Flow Shop*, o objetivo foi minimizar o *makespan*, resultando em valores ótimos iguais a 24 e 28 unidades de tempo, respectivamente. Os resultados foram exportados em arquivos CSV e visualizados por meio de gráficos de Gantt, permitindo avaliar a sequência ótima das operações, os períodos de ocupação das máquinas e os gargalos de cada formulação.

Keywords: Programação inteira mista, Scheduling, Machine Scheduling, Job Shop, Flow Shop, JuMP, HiGHS, Gráfico de Gantt

1. Introdução

Problemas de escalonamento de tarefas, ou *scheduling*, aparecem em diversos contextos de engenharia, produção, logística, computação e sistemas de manufatura. Em geral, o objetivo é definir quando cada tarefa deve ser executada e em qual ordem, respeitando restrições de precedência, disponibilidade de máquinas, tempos de processamento e prazos de entrega.

Neste trabalho, são analisadas três formulações clássicas. A primeira corresponde ao *Machine Scheduling* em máquina única, em que um conjunto de tarefas deve ser processado sem sobreposição, considerando tempos de liberação e prazos. A segunda corresponde ao *Job Shop Scheduling*, em que cada job possui uma rota própria de operações em diferentes máquinas. A terceira corresponde ao *Flow Shop Scheduling*, em que todos os jobs passam pelas mesmas máquinas na mesma ordem.

A resolução foi realizada por Programação Linear Inteira Mista, utilizando o ambiente JuMP e o solver HiGHS. Essa abordagem permite obter soluções ótimas para as instâncias avaliadas, diferentemente de heurísticas simples como FIFO, SPT ou EDD, que produzem sequências válidas, mas não garantem otimalidade global.

2. Metodologia

A metodologia adotada foi organizada em quatro etapas principais. Inicialmente, os dados de cada problema foram definidos em arquivos CSV. Em seguida, cada formulação foi implementada em Julia, utilizando variáveis contínuas para representar tempos de início e conclusão das operações e variáveis binárias para representar decisões de precedência. Na terceira etapa, os modelos foram resolvidos pelo HiGHS. Por fim, os resultados foram exportados em CSV e visualizados por gráficos de Gantt gerados em Python.

O uso de variáveis binárias é essencial nos três problemas, pois a principal dificuldade de scheduling está associada à escolha da ordem de execução das tarefas. Para duas tarefas ou operações que competem pela mesma máquina, o modelo precisa decidir se a primeira ocorre antes da segunda ou se a segunda ocorre antes da primeira. Essa decisão discreta transforma o problema em um modelo MILP.

A Tabela 1 resume os resultados obtidos nas três instâncias avaliadas.

Tabela 1: Resumo dos resultados computacionais obtidos.

Problema	Objetivo	Valor ótimo	Status
Machine Scheduling	Minimizar atraso total	16	OPTIMAL
Job Shop Scheduling	Minimizar makespan	24	OPTIMAL
Flow Shop Scheduling	Minimizar makespan	28	OPTIMAL

3. Capítulo 1: Machine Scheduling

3.1. Descrição do problema

O problema de *Machine Scheduling* avaliado considera uma única máquina e um conjunto de tarefas independentes. Cada tarefa possui um tempo de liberação, uma duração fixa e um prazo de entrega. A máquina pode processar apenas uma tarefa por vez, logo não pode haver sobreposição entre tarefas.

O objetivo adotado foi minimizar o atraso total, definido como a soma dos atrasos individuais das tarefas. Uma tarefa apresenta atraso quando seu tempo de conclusão excede o prazo de entrega.

3.2. Formulação MILP

Seja J o conjunto de tarefas. Para cada tarefa $j \in J$, são definidos os parâmetros:

$$r_j = \text{tempo de liberação da tarefa } j, \quad (1)$$

$$p_j = \text{tempo de processamento da tarefa } j, \quad (2)$$

$$d_j = \text{prazo de entrega da tarefa } j. \quad (3)$$

As variáveis contínuas são:

$$s_j = \text{tempo de início da tarefa } j, \quad (4)$$

$$C_j = \text{tempo de conclusão da tarefa } j, \quad (5)$$

$$T_j = \text{atraso da tarefa } j. \quad (6)$$

Para cada par de tarefas i e j , utiliza-se uma variável binária de precedência:

$$x_{ij} = \begin{cases} 1, & \text{se a tarefa } i \text{ é processada antes da tarefa } j, \\ 0, & \text{caso contrário.} \end{cases} \quad (7)$$

A função objetivo é dada por:

$$\min \sum_{j \in J} T_j. \quad (8)$$

As restrições principais são:

$$s_j \geq r_j, \quad \forall j \in J, \quad (9)$$

$$C_j = s_j + p_j, \quad \forall j \in J, \quad (10)$$

$$T_j \geq C_j - d_j, \quad \forall j \in J, \quad (11)$$

$$T_j \geq 0, \quad \forall j \in J. \quad (12)$$

A não sobreposição entre tarefas é imposta por restrições do tipo Big-M:

$$s_j \geq C_i - M(1 - x_{ij}), \quad \forall i, j \in J, i < j, \quad (13)$$

$$s_i \geq C_j - Mx_{ij}, \quad \forall i, j \in J, i < j. \quad (14)$$

3.3. Resultados obtidos

A sequência ótima encontrada para a instância foi:

$$E \rightarrow D \rightarrow A \rightarrow F \rightarrow B \rightarrow G \rightarrow C. \quad (15)$$

O atraso total ótimo foi igual a:

$$\sum_{j \in J} T_j = 16. \quad (16)$$

A Tabela 2 apresenta o cronograma obtido.

Tabela 2: Cronograma ótimo obtido para o problema de Machine Scheduling.

Tarefa	Início	Término	Duração	Atraso
E	0	2	2	0
D	2	6	4	0
A	6	11	5	1
F	11	14	3	0
B	14	20	6	0
G	20	22	2	0
C	22	30	8	15

A Figura 1 apresenta o gráfico de Gantt do resultado.

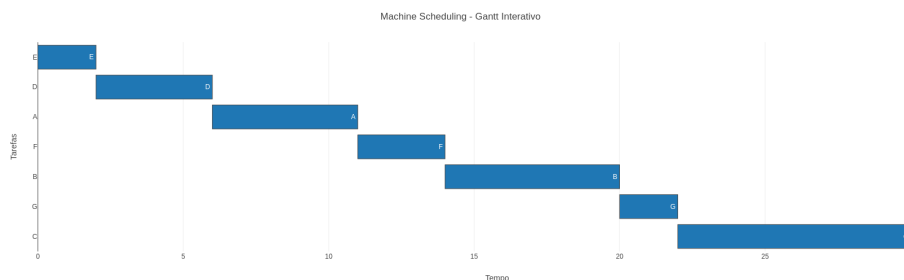


Figura 1: Gráfico de Gantt obtido para o problema de Machine Scheduling.

3.4. Discussão

O resultado mostra que o modelo priorizou uma sequência que reduz o atraso total global, e não necessariamente o atraso individual de todas as tarefas. A tarefa *C* concentrou a maior parcela do atraso, com 15 unidades de tempo, enquanto a tarefa *A* apresentou atraso igual a 1. Essa distribuição indica que o solver preferiu atrasar uma tarefa específica para preservar o desempenho global do cronograma.

Como o status da solução foi **OPTIMAL**, o resultado não deve ser interpretado como uma heurística. O solver certificou que não existe outra sequência viável, dentro da formulação proposta, com atraso total menor que 16.

4. Capítulo 2: Job Shop Scheduling

4.1. Descrição do problema

No *Job Shop Scheduling*, cada job é composto por uma sequência de operações. Cada operação deve ser processada em uma máquina específica, e a ordem interna das operações de cada job deve ser respeitada. Diferentemente do Flow Shop, os jobs podem possuir rotas distintas entre as máquinas.

O objetivo considerado foi minimizar o *makespan*, isto é, o instante de conclusão da última operação do cronograma.

4.2. Formulação MILP

Seja O o conjunto de operações. Para cada operação $o \in O$, define-se o tempo de processamento p_o e a máquina requerida m_o . As variáveis contínuas são o tempo de início s_o e o tempo de conclusão C_o . Define-se ainda a variável $C_{\max}$, que representa o *makespan*.

A função objetivo é:

$$\min C_{\max}. \quad (17)$$

As restrições de duração são:

$$C_o = s_o + p_o, \quad \forall o \in O. \quad (18)$$

Para operações consecutivas de um mesmo job, impõe-se:

$$s_{o+1} \geq C_o. \quad (19)$$

Para operações que utilizam a mesma máquina, aplica-se novamente uma lógica de precedência binária, impedindo sobreposição. Para duas operações a e b na mesma máquina:

$$s_b \geq C_a - M(1 - y_{ab}), \quad (20)$$

$$s_a \geq C_b - My_{ab}. \quad (21)$$

Por fim, o *makespan* é imposto por:

$$C_{\max} \geq C_o, \quad \forall o \in O. \quad (22)$$

4.3. Resultados obtidos

Para a instância analisada, o valor ótimo de *makespan* foi:

$$C_{\max} = 24. \quad (23)$$

A Tabela 3 apresenta o cronograma ótimo obtido.

Tabela 3: Cronograma ótimo obtido para o problema de Job Shop Scheduling.

Operação	Máquina	Início	Término	Duração	Job
J2_O1_M2	M2	0	8	8	J2
J3_O1_M3	M3	0	5	5	J3
J4_O1_M1	M1	0	5	5	J4
J1_O1_M1	M1	5	6	1	J1
J3_O2_M1	M1	6	10	4	J3
J1_O2_M2	M2	8	11	3	J1
J2_O2_M3	M3	8	13	5	J2
J3_O3_M2	M2	11	19	8	J3
J2_O3_M1	M1	13	23	10	J2
J4_O2_M3	M3	13	18	5	J4
J1_O3_M3	M3	18	24	6	J1
J4_O3_M2	M2	19	24	5	J4

A Figura 2 apresenta o gráfico de Gantt do cronograma.

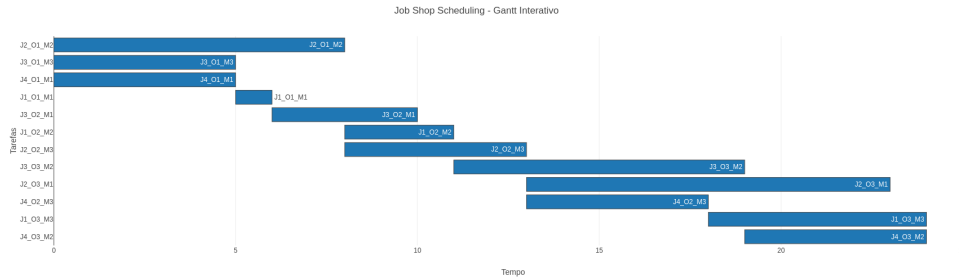


Figura 2: Gráfico de Gantt obtido para o problema de Job Shop Scheduling.

4.4. Discussão

O gráfico de Gantt evidencia a natureza combinatória do Job Shop. As operações são distribuídas em diferentes máquinas e precisam respeitar simultaneamente a ordem tecnológica dos jobs e a capacidade unitária das máquinas. O *makespan* ótimo igual a 24 indica que todas as operações foram concluídas até esse instante.

Observa-se que algumas máquinas permanecem ocupadas em intervalos críticos, enquanto outras apresentam períodos de ociosidade. Essa ociosidade não representa erro de modelagem; ela ocorre porque algumas operações só podem começar depois da conclusão de etapas anteriores do mesmo job ou após a liberação da máquina requerida.

5. Capítulo 3: Flow Shop Scheduling

5.1. Descrição do problema

No *Flow Shop Scheduling*, todos os jobs passam pelas mesmas máquinas e na mesma ordem. Assim, a principal decisão do modelo é determinar a sequência dos jobs na linha de produção. O objetivo adotado foi minimizar o *makespan*.

A instância analisada possui cinco jobs e três máquinas. A sequência ótima na primeira máquina foi:

$$J5 \rightarrow J4 \rightarrow J3 \rightarrow J2 \rightarrow J1. \quad (24)$$

5.2. Formulação MILP

Seja J o conjunto de jobs e K o conjunto de máquinas. Para cada job j e máquina k , define-se o tempo de processamento p_{jk} . As variáveis contínuas são:

$$s_{jk} = \text{tempo de início do job } j \text{ na máquina } k, \quad (25)$$

$$C_{jk} = \text{tempo de conclusão do job } j \text{ na máquina } k. \quad (26)$$

A função objetivo é:

$$\min C_{\max}. \quad (27)$$

As restrições de duração são:

$$C_{jk} = s_{jk} + p_{jk}, \quad \forall j \in J, k \in K. \quad (28)$$

Como todos os jobs seguem a mesma ordem de máquinas, impõe-se:

$$s_{j,k+1} \geq C_{jk}. \quad (29)$$

Além disso, em uma mesma máquina, dois jobs não podem ser processados simultaneamente. Para cada par de jobs i e j em uma máquina k , define-se uma variável binária de precedência z_{ij} :

$$s_{jk} \geq C_{ik} - M(1 - z_{ij}), \quad (30)$$

$$s_{ik} \geq C_{jk} - Mz_{ij}. \quad (31)$$

O *makespan* é definido por:

$$C_{\max} \geq C_{j,K}, \quad \forall j \in J. \quad (32)$$

5.3. Resultados obtidos

Para a instância avaliada, o *makespan* ótimo foi:

$$C_{\max} = 28. \quad (33)$$

A Tabela 4 apresenta o cronograma ótimo.

Tabela 4: Cronograma ótimo obtido para o problema de Flow Shop Scheduling.

Operação	Máquina	Início	Término	Duração	Job
J5_M1	M1	0	2	2	J5
J4_M1	M1	2	8	6	J4
J5_M2	M2	2	7	5	J5
J5_M3	M3	7	11	4	J5
J3_M1	M1	8	11	3	J3
J4_M2	M2	8	11	3	J4
J2_M1	M1	11	15	4	J2
J3_M2	M2	11	15	4	J3
J4_M3	M3	11	16	5	J4
J1_M1	M1	15	20	5	J1
J2_M2	M2	15	21	6	J2
J3_M3	M3	16	23	7	J3
J1_M2	M2	21	23	2	J1
J2_M3	M3	23	25	2	J2
J1_M3	M3	25	28	3	J1

A Figura 3 apresenta o gráfico de Gantt do resultado obtido.

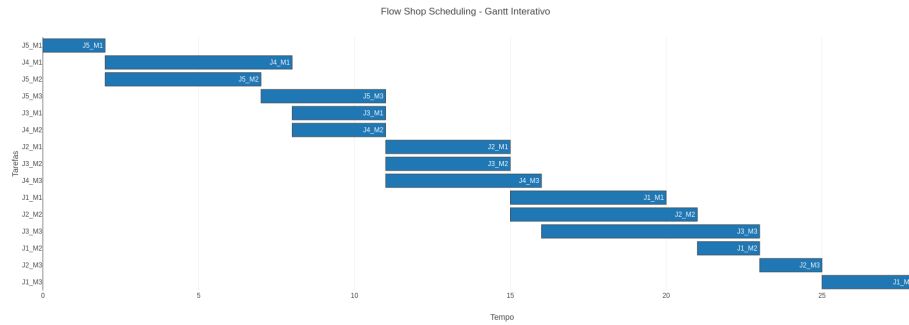


Figura 3: Gráfico de Gantt obtido para o problema de Flow Shop Scheduling.

5.4. Discussão

O resultado mostra que a sequência ótima inicia com o job $J5$, seguido por $J4$, $J3$, $J2$ e $J1$. Essa ordem reduz o tempo total de conclusão da linha, levando a um *makespan* de 28 unidades de tempo.

No Flow Shop, a mesma ordem de processamento tende a se propagar pelas máquinas, mas a existência de diferentes durações em cada etapa pode gerar intervalos de espera entre operações. O gráfico de Gantt permite visualizar esses intervalos e identificar como o fluxo de jobs atravessa a sequência de máquinas.

6. Análise Comparativa

A Tabela 5 compara as principais características dos três problemas avaliados.

Tabela 5: Comparação entre as três formulações avaliadas.

Problema	Estrutura	Decisão principal	Métrica otimizada
Machine Scheduling	Uma máquina	Ordem das tarefas	Atraso total
Job Shop	Rotas distintas	Ordem por máquina	Makespan
Flow Shop	Rota comum	Sequência dos jobs	Makespan

O Machine Scheduling é o caso mais simples do ponto de vista estrutural, pois envolve apenas uma máquina. Mesmo assim, a escolha da ordem das tarefas já exige variáveis binárias para garantir a não sobreposição. O Job Shop é mais complexo, pois cada job pode ter uma sequência própria de máquinas, gerando interdependências entre precedência tecnológica e disputa por recursos. O Flow Shop possui estrutura mais regular, pois todos os jobs seguem a mesma rota, mas ainda exige uma escolha combinatória da sequência ótima.

Em todos os casos, o uso de MILP permitiu obter soluções certificadas como ótimas pelo solver. Portanto, os resultados apresentados servem como referência para comparação com heurísticas construtivas ou meta-heurísticas em instâncias maiores.

7. Conclusões

Este trabalho apresentou a modelagem e a resolução computacional de três problemas clássicos de scheduling por Programação Linear Inteira Mista.

Os modelos foram implementados em Julia com JuMP e resolvidos com HiGHS.

No problema de Machine Scheduling, a solução ótima apresentou atraso total igual a 16, com sequência $E \rightarrow D \rightarrow A \rightarrow F \rightarrow B \rightarrow G \rightarrow C$. No problema de Job Shop Scheduling, o *makespan* ótimo foi igual a 24. No problema de Flow Shop Scheduling, o *makespan* ótimo foi igual a 28, com sequência inicial $J5 \rightarrow J4 \rightarrow J3 \rightarrow J2 \rightarrow J1$.

Os gráficos de Gantt complementaram a análise ao permitir visualizar a ocupação das máquinas e a ordem das operações no tempo. A combinação entre modelagem MILP, exportação em CSV e visualização interativa mostrou-se adequada para análise computacional dos problemas propostos.

Como trabalhos futuros, recomenda-se comparar os resultados ótimos com heurísticas como FIFO, SPT, EDD e NEH, além de avaliar instâncias maiores com limites de tempo, gaps de otimalidade e diferentes configurações do solver.

Arquivos gerados

Os dados de saída utilizados neste artigo estão organizados nos arquivos CSV do projeto:

- `data/machine_scheduling_schedule.csv`;
- `data/job_shop_schedule.csv`;
- `data/flow_shop_schedule.csv`.